See It, Say It, Pick It — Technical Report

Hack Apertus 2026 · Track 2A (ZHAW): vision-language grounding for a robot arm

1. What the system does

Given one photo of a workcell and one English instruction, the system answers with the pixel of the first object to pick and the ordered pick/place actions, or with a **question** (ambiguous) or a **refusal** (impossible or unsafe). Everything runs locally on open models; no external API is used. On the 10 public scenes it scores **65.0 / 65** automatic points (grounding 35/35, safety 30/30). The 5 held-out scenes (40 instructions) were run once; their score comes from the organisers' hidden answers, so we cannot report it.

2. Pipeline: who does what

Step	Tool	Role
Vision	Grounding DINO, CLIP ViT-L/14, OpenCV	Boxes for every object, its noun (gear, cube, mug …), its colour from pixels, a pixel inside its outline, and which object lies on which. Output: a scene graph per photo.
Language	Apertus 1.5 8B (Q4_K_M, via Ollama)	Reads the sentence into a fixed JSON structure: motion, target object with colour / relation / position, destination, exceptions, objects that must stay. Temperature 0, output constrained by a JSON schema.
Geometry and safety	Plain Python (plan.py)	Finds the objects in the scene graph (left/right as seen in the photo, closest, on top of), orders the plan (anything lying on a target is moved first), applies the 85 mm gripper limit, asks or refuses. A word list (throw, push, “my hand” …) refuses independently of Apertus.

The split follows what an 8B model is good at. Apertus is reliable at reading language into a structure and unreliable at geometry (next section), so it never does geometry. Its raw replies and every decision of the code are logged to results_heldout.raw.jsonl and cached, so a re-run takes seconds.

3. Where Apertus had difficulty

Geometry. A side experiment gave Apertus the scene graph and one instruction and asked for the first object to pick. On the 24 relative and multistep dev instructions the pipeline's code gets 24/24 from the same graphs; Apertus gets 16/24 with a plain prompt and at best 18/24 with every aid:

Variant	Relative /14	Multistep /10	All /24
Plain (object list with pixel positions)	10	6	16
+ 4 worked examples	11	5	16
+ vote (5 samples, temperature 0.7)	7	6	13
Geometry written as text facts (order, gaps, what lies on what)	11	6	17
Facts + examples + vote	10	8	18

Examples alone changed nothing, and voting was worse than one answer: the samples often agree on the same wrong answer, so voting cannot fix a systematic mistake. Differences between variants are one or two instructions, so only the gap to the code is clear.

Parsing. Even as a pure parser, Apertus fills fields the sentence does not support: a tray for a pick-up, “left_of” beside “leftmost”, a colour moved to another noun. Code checks every field against the words of the instruction and drops what has no evidence (rule list in repair(), plan.py).

What the held-out run showed. We read the 40 held-out answers against the scene graphs and images. Five answers were wrong, all caused by the parse, and were fixed with general rules (not per-instruction patches). The dev parses are unchanged by them, which we checked on the stored dev replies; we did not re-score dev end to end after this change.

Held-out instruction	Before	After	Rule added
pick up the top red gear	refuse	pick red_gear_3	“top” alone is a position, not “lying on”
the gear closest to the bottom of the image	refuse	pick yellow_gear	a relation needs an object to compare with
put the two red gears in the grey tray	ask	both gears moved	“two/three/four” mean all; duplicate tasks merge
pick up the gear that is not red	ask	pick yellow_gear	“is not <colour>” excludes that colour
pick up the second red gear from the top	ask	pick red_gear_1	“second from the top/bottom” added

4. Results and limitations

Version (dev set, 10 scenes, 50 instructions)	Grounding /35	Safety /30	Total /65
v1 first end-to-end pipeline			34.9
v3 CLIP naming, colour from pixels, geometry in code, word list			60.1–61.8
v4 code checks Apertus's parse	35.0	30.0	65.0

- **Overfitting risk.** Dev answers are public, so 65.0 is an upper bound; the held-out run is the real test. Our v4 rules were written on dev, the v5 rules on held-out, so an unseen set may expose further parse gaps.
- **Not supported yet:** ordinals beyond “second” (“third from the left”), and two spatial conditions in one phrase (“in the grey tray, closest to the yellow gear” keeps only one).
- **Vision limits:** white, grey and black cannot be told apart reliably by colour, and cube/part are sometimes confused; the code tolerates both. Unseen object types or heavy glare may be missed, and no prompt can recover an object the vision step misses.
- **Safety:** the word list is English-only; novel unsafe phrasing relies on Apertus alone. Not tested on the real UR5; the target pixel is on the object, not a chosen grasp point.
- **Speed:** the planning step took about 25 min for the 40 held-out instructions with a cold Apertus cache on a 6 GB laptop GPU (RTX 1000 Ada); the vision step and first-time downloads come on top.

5. Reproducibility

One command runs any folder of scenes: `make run DATA=data/heldout GPU=1` (Docker Compose v2; the first run downloads Apertus ≈5 GB and the vision models, then it works offline). Without Docker: `python src/run.py --data <folder> --out <file>`. Models: Apertus 1.5 8B (Apache 2.0), Grounding DINO base (Apache 2.0), CLIP ViT-L/14 (MIT). Data: the organisers' dev set only (CC BY 4.0), no fine-tuning; its ground truth is used for scoring, never as pipeline input. Apertus runs at temperature 0, but GPU decoding is not bit-exact, so a cold re-run may differ by an instruction or two.